Engineering Statistics and Data Analysis

Engineering Computation with Python — Chapter 14

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

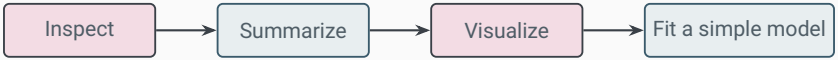
Table of Contents

1. Understanding Engineering Measurements
2. Basic Statistical Analysis
3. Introduction to Data Analytics
4. pandas for Data Analytics
5. Grouping, Cleaning and Outliers
6. Data Analysis and Visualization
7. Machine Learning (Optional Preview)

© Muním Zabidi

The Core Path Through This Chapter

This chapter covers a lot of ground, so it helps to know the destination first.



What each step means

Inspect: what columns and rows does the dataset have, and are any values missing? *Summarize*: mean, spread, outliers. *Visualize*: does the summary match what a plot shows? *Fit*: can one quantity predict another?

Machine learning is an optional preview

The closing section on ML classifiers previews where this path leads in later courses. It is not required to complete the core path above.

© Muním Zabidi

Table of Contents

1. Understanding Engineering Measurements
2. Basic Statistical Analysis
3. Introduction to Data Analytics
4. pandas for Data Analytics
5. Grouping, Cleaning and Outliers
6. Data Analysis and Visualization
7. Machine Learning (Optional Preview)

Population vs. Sample

A population is every possible measurement that could be made. A sample is the subset you actually recorded.

Example

A factory produces a million resistors; you measure fifty. The million is the population, the fifty are your sample. Almost all engineering statistics works from samples — measuring the whole population is usually impossible.

Why it matters computationally: the *sample* variance divides by $N - 1$ rather than N, to compensate for a small sample underestimating the true spread.

Population vs. Sample Variance in NumPy

```python
import numpy as np

readings = np.array([98.2, 101.5, 99.7, 100.3,
                     102.1, 99.1, 100.8])  # ohms

print("Population variance:", np.var(readings))
print("Sample variance:    ", np.var(readings, ddof=1))
```

```
Population variance:  1.588
Sample variance:  1.853
```

NumPy computes the population form by default (`ddof=0`) and the sample form when told to (`ddof=1`).

Mean, Range, Variance, Standard Deviation

Four quantities describe most measurement sets adequately.

Mean

$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$ — the best single estimate of the true value.

Variance

$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$ — average squared distance from the mean.

Range

Largest value minus smallest — crude, but instantly informative about outliers.

Standard deviation

$\sigma = \sqrt{\sigma^2}$ — more useful in practice since it shares the same units as the measurement.

© Muním Zabidi

Worked Example: Seven Resistors

```
readings = np.array([98.2, 101.5, 99.7, 100.3,
                     102.1, 99.1, 100.8])  # ohms

print(f"Mean:   {np.mean(readings):.2f} ohms")
print(f"Range:  {np.ptp(readings):.2f} ohms")
print(f"Std:    {np.std(readings, ddof=1):.2f} ohms")
```

```
Mean:  100.24 ohms  Range:  3.90 ohms  Std:  1.36 ohms
```

Interpret

Seven 100 Ω (5% tolerance) resistors must fall in 95–105 Ω. Every reading here does — this batch passes.

Measurement Error: Absolute vs. Relative

When the true value is known independently (a calibrated reference, say), measurement quality can be quantified directly.

$$e = x_{\text{measured}} - x_{\text{true}}, \qquad \%\,\text{error} = \frac{|e|}{x_{\text{true}}} \times 100.$$

Why relative error matters

An absolute error of $0.243\,\Omega$ is excellent on a $100\,\Omega$ resistor, but catastrophic on a $1\,\Omega$ shunt, where it is a 24% error. The relative form lets instruments of different ranges be compared fairly.

© Munim Zabidi

Summarizing the running bench dataset statistically reveals its corrupted reading immediately.

`temp.mean()` = 28.28 C `temp.median()` = 27.71 C `temp.max()` = 98.40 C `temp.std()` = 7.06 C

Interpret

Mean and median are close, but the max is wildly out of keeping with a board that sits near room temperature. This is the signature of a single corrupted sample: the median barely moves, while the mean and standard deviation are inflated. A mean far from the median is often the first clue of an outlier — the IQR rule (later this chapter) identifies it precisely.

© Muním Zabidi

Table of Contents

1 Understanding Engineering Measurements

2 **Basic Statistical Analysis**

3 Introduction to Data Analytics

4 pandas for Data Analytics

5 Grouping, Cleaning and Outliers

6 Data Analysis and Visualization

7 Machine Learning (Optional Preview)

NumPy/SciPy Statistical Functions

Function	Description
`np.max()` / `np.min()`	Maximum / smallest value
`np.mean()`	Average value
`np.median()`	Median value
`stats.mode()`	Most frequent value (`scipy.stats`)
`np.std()` / `np.var()`	Standard deviation / variance

Choosing an Axis: Rows vs. Columns

When performing statistics on a matrix, the `axis` parameter decides the direction.

```
x = np.array([[1, 3], [2, 5], [6, 4]])

print(np.mean(x, axis=0))  # down columns: [3. 4.]
print(np.mean(x, axis=1))  # across rows: [2. 3.5 5.]
print(np.mean(x))          # every element: 3.5
```

Rule of thumb

`axis=0` collapses rows (one result per column). `axis=1` collapses columns (one result per row). No `axis` → one scalar over the whole array.

Worked Example: Temperature Across Three Cities

A user-defined `datastat()` function computes mean, variance, median, max, and min column-wise (`axis=0`) for three cities' temperature readings, then optionally plots them.

```
Dataset 1:  avg 11.97   Dataset 2:  avg 8.23   Dataset 3:  avg 19.87
```

The pattern, not the code

Writing a small reusable statistics function once, rather than repeating `np.mean()/np.var()` calls inline, is what makes this scale to many datasets.

Table of Contents

1 Understanding Engineering Measurements

2 Basic Statistical Analysis

3 Introduction to Data Analytics

4 pandas for Data Analytics

5 Grouping, Cleaning and Outliers

6 Data Analysis and Visualization

7 Machine Learning (Optional Preview)

What Is Data Analytics?

Data analytics examines, cleans, transforms, and interprets data to extract insights and inform decisions. Raw data, by itself, usually means nothing.

Benefits

Informed decisions, efficiency, competitive advantage, risk management, cost reduction, predictive maintenance.

Domains

Business intelligence, healthcare, finance, manufacturing, education, energy, public policy.

© Munim Zabidi

The first step is always to define the problem:

- Why are we losing customers?
- Which factors hurt the customer experience?
- How can we boost retention while cutting costs?

Then: find the data

Once the problem is defined, determine the best data sources — sensors, databases, surveys, logs — to help solve it.

© Muním Zabidi

Types of Data

Structured	Unstructured	Semi-structured
Predefined format, rows and columns. *E.g.* dates, transaction records.	No predefined format. *E.g.* documents, images, audio.	Some structure, no rigid schema. *E.g.* JSON, server logs.

Most engineering measurement data in this course is structured — the kind pandas handles directly.

Table of Contents

1. Understanding Engineering Measurements
2. Basic Statistical Analysis
3. Introduction to Data Analytics
4. **pandas for Data Analytics**
5. Grouping, Cleaning and Outliers
6. Data Analysis and Visualization
7. Machine Learning (Optional Preview)

Why pandas?

pandas is the standard tool for loading, exploring, cleaning, and analyzing structured (tabular) data in Python.

Core features

Load from CSV/Excel/SQL; explore with `.head()`, `.shape`, `.info()`; summarize with `.describe()`; filter rows/columns.

Further features

Missing data (`.isnull()`, `.fillna()`); grouping (`.groupby()`); merging (`.merge()`, `.concat()`).

First Look at a Dataset

```python
import pandas as pd

df = pd.read_csv('employees.csv')
print(df.head())    # first 5 rows
print(df.shape)     # (1000, 8)
print(df.describe())  # count, mean, std, min, 25/50/75%, max
```

The routine first pass

`.head()`, `.shape`, and `.describe()` together answer: what does this dataset look like, how big is it, and what do the numeric columns typically contain?

What Is a DataFrame?

A pandas DataFrame is a 2-D, labeled table: rows and columns, each column its own data type, built from dictionaries, arrays, CSV/Excel files, or SQL.

Why dictionaries matter here

A DataFrame is most easily built from a dictionary — each key becomes a column name, each value the data beneath it. This is exactly why the dictionaries from Chapter 3 are worth knowing.

© Muním Zabidi

Building a DataFrame

```
Sensor = ['TMP-01', 'TMP-02', 'TMP-03', 'TMP-04']
Readings = [10, 12, 16, 23]

df = pd.DataFrame({'Sensor': Sensor, 'Readings': Readings})
print(df)
print("Mean:", df['Readings'].mean())
```

```
Mean of Readings:  16.0
```

Column access (`df['Readings']`) behaves like a labeled NumPy array — all the usual statistics methods apply.

Table of Contents

1. Understanding Engineering Measurements
2. Basic Statistical Analysis
3. Introduction to Data Analytics
4. pandas for Data Analytics
5. **Grouping, Cleaning and Outliers**
6. Data Analysis and Visualization
7. Machine Learning (Optional Preview)

Data Aggregation with `groupby`

Summarizing *by category* — average power per component type, say — is one of the most common tasks on a loaded dataset.

```
data = {'component':   ['Resistor','Resistor','Capacitor',
                        'Capacitor','Inductor','Inductor'],
        'power_watts': [2.1, 2.4, 1.5, 1.8, 3.2, 3.6]}
df = pd.DataFrame(data)

summary = df.groupby('component')['power_watts'].agg(['mean','sum'])
print(summary)
```

```
Capacitor 1.65 / 3.3  Inductor 3.40 / 6.8  Resistor 2.25 / 4.5
```

Raw data often contains errors, missing values, and noise. Cleaning transforms it into something usable:

Reduction	Transformation	Integration
Remove irrelevant data.	Scale data for modeling compatibility.	Combine multiple sources into one dataset.

The process begins with inspection: find missing values, outliers, and inconsistencies.

© Muním Zabidi

Finding Missing Values

```
data = {'Node': ['ADC-01','ADC-02','ADC-03',None,'ADC-06'],
        'Samples': [10, None, 16, 18, 20],
        'Voltage': [2.5, 3.0, None, 3.2, 2.9]}
df = pd.DataFrame(data)

missing = df.isnull()
print(df.isnull().sum())   # count per column
```

```
Node:  1  Samples:  1  Voltage:  1
```

`.isnull()` returns a same-shaped DataFrame of `True`/`False`; missing values appear as `NaN` (numeric), `NaT` (datetime), or `None` (object).

Detecting Unusual Measurements

An outlier is a reading that differs significantly from the rest — it can distort statistics, skew models, or signal a genuine rare event.

A caution worth repeating

These methods detect unusual observations; they do not prove an observation is wrong. A statistical test cannot tell a sensor fault from a rare but real event — it can only point out where to look.

Two common methods: the **Z-score** method ($|Z| > 3$) and the **IQR (Tukey) method**, covered next.

Quartiles

Q_1 (25%), Q_2 (median, 50%), Q_3 (75%) — quarters of the sorted data.

Tukey's fences (1.5×IQR rule)

$$\text{lower} = Q_1 - 1.5\,\text{IQR}$$
$$\text{upper} = Q_3 + 1.5\,\text{IQR}$$

$$\text{IQR} = Q_3 - Q_1$$

Values outside [lower, upper] are flagged. The multiplier 1.5 can be adjusted for more or less sensitivity.

© Munim Zabidi

Worked Example: Computing the Bounds

Data: `[2, 10, 12, 18, 20, 23, 100]` (sorted).

$$Q_1 = 11, \qquad Q_3 = 21.5, \qquad \text{IQR} = 10.5$$

$$\text{lower} = 11 - 1.5(10.5) = -4.75, \qquad \text{upper} = 21.5 + 1.5(10.5) = 37.25$$

Result

No value is below -4.75. The value 100 exceeds 37.25 — flagged as an outlier.

Flagging and Handling Outliers

```
Q1 = df['Samples'].quantile(0.25)
Q3 = df['Samples'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR

df['Outlier'] = (df['Samples'] < lower) | (df['Samples'] > upper)
df.loc[df['Outlier'], 'Samples'] = np.nan   # or: replace with median
```

Two common fixes once flagged

Replace with `NaN` (then treat as missing data), or replace with the column median — a single outlier barely moves the median, making it a safe stand-in.

A Boxplot Shows the Same Test Graphically

- Box spans Q_1 to Q_3 (middle 50%).
- Line inside the box is the median.
- Whiskers extend to the Tukey bounds.
- Points beyond the whiskers are outliers.

`plt.boxplot(samples, vert=False)` draws exactly this — a visual cross-check on the numeric IQR calculation.

Missing data and outliers are the two issues that most commonly compromise an analysis — both require deliberate, documented handling.

© Muním Zabidi

Table of Contents

1 Understanding Engineering Measurements

2 Basic Statistical Analysis

3 Introduction to Data Analytics

4 pandas for Data Analytics

5 Grouping, Cleaning and Outliers

6 **Data Analysis and Visualization**

7 Machine Learning (Optional Preview)

Three Levels of Data Analysis

Type	Key Question	Focus
Descriptive	What happened?	Summarize historical data, find patterns
Predictive	What could happen?	Forecast from historical patterns
Prescriptive	What's the best action?	Choose the best course of action

Univariate describes a single variable (mean, median, mode). **Bivariate** examines the relationship between two.

```
heights = [157, 163, 176, 152]
print("Mean:", np.mean(heights))     # univariate

plt.scatter(temperature, sales)      # bivariate: relationship
```

Together, these summarize large datasets, reveal patterns, and flag issues *before* any modeling is attempted.

© Muním Zabidi

```python
from sklearn.linear_model import LinearRegression

X = [[26], [30], [35], [40]]   # temperature
y = [1000, 2500, 3500, 4500]   # sales

model = LinearRegression()
model.fit(X, y)
print(model.predict([[32]]))   # Predicted sales: [2693.0]
```

What just happened

The model learned the best-fit straight line relating temperature to sales — a typical example of supervised learning with a single feature.

© Munim Zabidi

Prescriptive analytics recommends actions, not just predictions — `if predicted_sales > 5000: increase production`.

Visualizing the dataset, not the signal

Once data is loaded, cleaned, and aggregated, a chart is usually the fastest way to communicate what was found. A bar chart compares categories (e.g. a `groupby` summary); a scatter plot shows a relationship between two variables.

Table of Contents

1. Understanding Engineering Measurements
2. Basic Statistical Analysis
3. Introduction to Data Analytics
4. pandas for Data Analytics
5. Grouping, Cleaning and Outliers
6. Data Analysis and Visualization
7. **Machine Learning (Optional Preview)**

Before you continue

This section previews where the core path leads in later courses. It is not required to understand the rest of this course — treat it as a survey, not a deep dive.

Predictive analytics uses machine learning: models learn from historical data and are used for prediction and classification, without being explicitly programmed for the specific task.

© Muním Zabidi

Regression vs. Classification

Predictive models divide into two kinds, by what they produce:

Regression

Predicts a quantity — a temperature, a voltage, a time constant.
"How hot will this motor get?"

Classification

Predicts a category — healthy or faulty, pass or fail.
"Is this motor overheating?"

A Simple Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

X = [[1], [2], [3], [4]]
y = ['low', 'low', 'high', 'high']

model = DecisionTreeClassifier()
model.fit(X, y)
print(model.predict([[3]])[0])   # high
```

A model has no sense of its own limits

Asked to classify 99 — far outside its training data — it still returns `high` regardless, since its only rule is “above 2.5”. Judging when a question falls outside the model’s experience remains the engineer’s job.

The General Machine Learning Workflow



Why training and testing sets are split

A model evaluated only on the data it memorized tells you nothing about how it performs on new measurements — always hold out a test set.

© Muním Zabidi

Three Types of Machine Learning

Supervised	Unsupervised	Reinforcement
Learns from labeled training data. *E.g.* regression, classification.	Learns from unlabeled data, finding patterns on its own.	Learns through feedback, reinforcing correct responses.

This course's examples are all supervised — regression and decision-tree classification.

Where This Leaves You

- The core path — inspect, summarize, visualize, fit a simple model — is what every engineering dataset needs, regardless of whether machine learning is involved.
- `LinearRegression` and `DecisionTreeClassifier` are two simple, inspectable starting points, not the final word.
- Neither can be trusted outside the range of data it was trained on: machine learning is one more engineering tool, not a substitute for understanding the problem.

This concludes the taught chapters of the course.

© Muním Zabidi